

SUÍTE DE TESTES UNITÁRIOS — SISTEMA REGRAPHIK

Suíte de testes das camadas API e APPWPF — Projeto ReGraphikApp

NOVA LIMA — MG
2026

SUÍTE DE TESTES UNITÁRIOS — SISTEMA REGRAPHIK

Relatório Técnico apresentado como pré-requisito para conclusão do Curso Superior de Tecnologia em Desenvolvimento de Sistemas, na CFP Escola de Programação e Robótica de Nova Lima, elaborado sob orientação do Instrutor Frederico Martins Aguiar.

Aluno: Bruno Maia Santos

Turma: Desenvolvimento de Sistemas

Projeto: ReGraphikApp

Componentes: InputSanitizationService, UsuarioController, ResiduoController, PontosColetaController (API); ValidacaoCpfService, UsuarioSessaoService, CadastroViewModel, BaseViewModel (APPWPF)

Tecnologia: C# / .NET 8.0

Framework: xUnit

SUMÁRIO

1. Suíte de testes unitários desenvolvida	4
1.1. API — Services (InputSanitizationService)	4
1.2. API — Controllers	9
1.3. APPWPF — Services	11
1.4. APPWPF — ViewModels	16
2. Estrutura dos testes	20
3. Documentação da estratégia utilizada	20
4. Explicação dos casos de teste	21
5. Resultado final dos testes	22
6. Identificação clara dos componentes escolhidos	23
7. Evidência de contribuição no projeto	23
8. Considerações finais	23
9. Checklist de evidências	24

1 SUÍTE DE TESTES UNITÁRIOS DESENVOLVIDA

A suíte de testes unitários foi desenvolvida individualmente para os componentes das camadas de API e APPWPF do projeto ReGraphikApp. A implementação utiliza C#, .NET 8.0 e o framework xUnit.

A suíte foi estruturada para verificar diferentes comportamentos dos componentes escolhidos, incluindo cenários de sucesso, entradas inválidas, valores limites, segurança, sanitização, validação de objetos, respostas HTTP de controllers e comportamento de ViewModels. Ao todo, foram considerados 42 casos de teste, distribuídos em quatro classes de teste.

Cada teste segue a organização Arrange, Act e Assert, permitindo separar a preparação dos dados, a execução do método e a verificação do resultado esperado.

1.1 API — Services (InputSanitizationService)

Os 15 casos de teste implementados para o componente InputSanitizationService estão descritos a seguir, agrupados pelo método testado. Cada caso apresenta o objetivo do teste e o respectivo código-fonte extraído da classe ApiServiceTests.cs.

IdEhSeguro()

- **CT01 — ID Válido**

Objetivo: Verificar se um identificador dentro das regras é aceito.

```
/// <summary>
/// Cenário 1: ID válido - Retorna true
/// </summary>
[Fact]
0 referências
public void IdEhSeguro_IdValido_DeveRetornarTrue()
{
    // Arrange
    var id = "abc123-usuario";

    // Act
    var resultado = InputSanitizationService.IdEhSeguro(id);

    // Assert
    Assert.True(resultado);
}
```

- **CT02 — ID Nulo**

Objetivo: Verificar o tratamento de identificador nulo.

```
/// <summary>
/// Cenário 2: ID nulo - Retorna false
/// </summary>
[Fact]
0 referências
public void IdEhSeguro_IdNulo_DeveRetornarFalse()
{
    // Arrange
    string? id = null;

    // Act
    var resultado = InputSanitizationService.IdEhSeguro(id);

    // Assert
    Assert.False(resultado);
}
```

- **CT03 — ID Vazio**

Objetivo: Verificar o tratamento de identificador vazio.

```

/// <summary>
/// Cenário 3: ID vazio - Retorna false
/// </summary>
[Fact]
0 referências
public void IdEhSeguro_IdVazio_DeveRetornarFalse()
{
    // Arrange
    var id = "";

    // Act
    var resultado = InputSanitizationService.IdEhSeguro(id);

    // Assert
    Assert.False(resultado);
}

```

- **CT04 — Path Traversal**

Objetivo: Verificar a rejeição de tentativa de Path Traversal.

- **CT05 — Caractere Inválido do Firebase**

Objetivo: Verificar a rejeição de caractere não permitido.

```

/// <summary>
/// Cenário 4: ID com path traversal - Retorna false
/// </summary>
[Fact]
0 referências
public void IdEhSeguro_PathTraversal_DeveRetornarFalse()
{
    // Arrange
    var id = "../admin";

    // Act
    var resultado = InputSanitizationService.IdEhSeguro(id);

    // Assert
    Assert.False(resultado);
}

```

- **CT06 — ID Maior que 128 Caracteres**

Objetivo: Verificar o limite máximo definido para o identificador.

```

/// <summary>
/// Cenário 6: ID acima de 128 caracteres - Retorna false
/// </summary>
[Fact]
0 referências
public void IdEhSeguro_IdMaiorQue128Caracteres_DeveRetornarFalse()
{
    // Arrange
    var id = new string('a', 129);

    // Act
    var resultado = InputSanitizationService.IdEhSeguro(id);

    // Assert
    Assert.False(resultado);
}

```

SanitizarTexto()

- **CT07 — Texto com HTML**

Objetivo: Verificar a remoção das tags HTML, preservando o conteúdo textual.

```

/// <summary>
/// Cenário 7: Texto contendo tags HTML - Remove as tags e preserva o conteúdo textual
/// </summary>
[Fact]
0 referências
public void SanitizarTexto_ComHtml_DeveRemoverTags()
{
    // Arrange
    var texto = "Olá <script>alert('x')</script> Bruno";

    // Act
    var resultado = InputSanitizationService.SanitizarTexto(texto);

    // Assert
    Assert.Equal("Olá alert('x') Bruno", resultado);
}

```

- **CT08 — Texto Nulo**

Objetivo: Verificar o tratamento de texto nulo.

```

/// <summary>
/// Cenário 8: Texto nulo - Retorna string vazia
/// </summary>
[Fact]
0 referências
public void SanitizarTexto_TextoNulo_DeveRetornarVazio()
{
    // Arrange
    string? texto = null;

    // Act
    var resultado = InputSanitizationService.SanitizarTexto(texto);

    // Assert
    Assert.Equal(string.Empty, resultado);
}

```

- **CT09 — Texto Acima do Limite**

Objetivo: Verificar o truncamento de texto acima do limite definido.

```

/// <summary>
/// Cenário 9: Texto acima do limite - Deve truncar
/// </summary>
[Fact]
0 referências
public void SanitizarTexto_TextoMaiorQueLimite_DeveTruncar()
{
    // Arrange
    var texto = "ABCDEFGHJIJ";

    // Act
    var resultado = InputSanitizationService.SanitizarTexto(texto, 5);

    // Assert
    Assert.Equal("ABCDE", resultado);
}

```

ValidarResiduo()

- **CT10 — Resíduo Válido**

Objetivo: Verificar se dados válidos não geram erros de validação.

```

/// <summary>
/// Cenário 10: Resíduo válido - Não deve retornar erros
/// </summary>
[Fact]
0 referências
public void ValidarResiduo_ResiduoValido_DeveRetornarListaVazia()
{
    // Arrange
    var residuo = new Residuo
    {
        TipoResiduo = "Papel Offset",
        Origem = "Produção",
        Quantidade = 10,
        Especificacao = "Fosco",
        Observacao = "Material reaproveitável"
    };

    // Act
    var erros = InputSanitizationService.ValidarResiduo(residuo);

    // Assert
    Assert.Empty(erros);
}

```

- **CT11 — Dados Obrigatórios Inválidos**

Objetivo: Verificar as validações de campos obrigatórios (tipo, origem e quantidade).

```

/// <summary>
/// Cenário 11: Tipo e origem vazios, quantidade inválida - Deve retornar erros
/// </summary>
[Fact]
0 referências
public void ValidarResiduo_DadosObrigatoriosInvalidos_DeveRetornarErros()
{
    // Arrange
    var residuo = new Residuo
    {
        TipoResiduo = "",
        Origem = "",
        Quantidade = 0
    };

    // Act
    var erros = InputSanitizationService.ValidarResiduo(residuo);

    // Assert
    Assert.Equal(3, erros.Count);
    Assert.Contains("tipo_residuo", erros[0]);
    Assert.Contains("origem", erros[1]);
    Assert.Contains("quantidade", erros[2]);
}

```

ValidarUsuario()

- **CT12 — Usuário Válido**

Objetivo: Verificar se um usuário válido passa pelas regras de validação.

```

/// <summary>
/// Cenário 12: Usuário válido - Não deve retornar erros
/// </summary>
[Fact]
0 referências
public void ValidarUsuario_UsuarioValido_DeveRetornarListaVazia()
{
    // Arrange
    var usuario = new Usuario
    {
        Nome = "Bruno Maia",
        Email = "bruno@gmail.com",
        Login = "bruno",
        Senha = "123456"
    };

    // Act
    var erros = InputSanitizationService.ValidarUsuario(usuario);

    // Assert
    Assert.Empty(erros);
}

```

- **CT13 — E-mail Inválido**

Objetivo: Verificar a validação do formato do e-mail.

```

/// <summary>
/// Cenário 13: E-mail inválido - Deve retornar erro
/// </summary>
[Fact]
0 referências
public void ValidarUsuario_EmailInvalido_DeveRetornarErro()
{
    // Arrange
    var usuario = new Usuario
    {
        Nome = "Bruno Maia",
        Email = "email-invalido",
        Login = "bruno",
        Senha = "123456"
    };

    // Act
    var erros = InputSanitizationService.ValidarUsuario(usuario);

    // Assert
    Assert.Contains(erros, erro => erro.Contains("email"));
}

```

- **CT14 — Login Muito Curto**

Objetivo: Verificar o limite mínimo de caracteres do login.

```

/// <summary>
/// Cenário 14: Login curto - Deve retornar erro
/// </summary>
[Fact]
0 referências
public void ValidarUsuario_LoginMuitoCurto_DeveRetornarErro()
{
    // Arrange
    var usuario = new Usuario
    {
        Nome = "Bruno Maia",
        Email = "bruno@gmail.com",
        Login = "ab",
        Senha = "123456"
    };

    // Act
    var erros = InputSanitizationService.ValidarUsuario(usuario);

    // Assert
    Assert.Contains(erros, erro => erro.Contains("login"));
}

```

- **CT15 — Senha Muito Curta**

Objetivo: Verificar o limite mínimo de caracteres da senha.

```
/// <summary>
/// Cenário 15: Senha curta - Deve retornar erro
/// </summary>
[Fact]
0 referências
public void ValidarUsuario_SenhaMuitoCurta_DeveRetornarErro()
{
    // Arrange
    var usuario = new Usuario
    {
        Nome = "Bruno Maia",
        Email = "bruno@gmail.com",
        Login = "bruno",
        Senha = "12345"
    };

    // Act
    var erros = InputSanitizationService.ValidarUsuario(usuario);

    // Assert
    Assert.Contains(erros, erro => erro.Contains("senha"));
}
```

1.2 API — Controllers

Os 6 casos de teste a seguir verificam o comportamento dos controllers da API diante de identificadores inseguros e parâmetros obrigatórios ausentes, garantindo que a camada HTTP rejeite entradas indevidas antes de acionar a lógica de negócio. Código-fonte extraído da classe ApiControllerTests.cs.

UsuarioController

- **CT16 — GET Usuário com ID Inseguro**

Objetivo: Verificar se o controller rejeita, com BadRequest, uma requisição GET contendo tentativa de path traversal no ID.

```
/// <summary>
/// Cenário 16: ID inseguro no GET - Deve retornar BadRequest
/// </summary>
[Fact]
0 referências
public async Task UsuarioController_GetById_IdInseguro_DeveRetornarBadRequest()
{
    // Arrange
    var controller = CriarUsuarioController();

    // Act
    var resultado = await controller.GetById("../admin");

    // Assert
    var badRequest = Assert.IsType<BadRequestObjectResult>(resultado);
    Assert.Equal(400, badRequest.StatusCode);
}
```

- **CT17 — DELETE Usuário com ID Inseguro**

Objetivo: Verificar se o controller rejeita, com BadRequest, uma requisição DELETE contendo ID inseguro.

```

/// <summary>
/// Cenário 17: ID inseguro no DELETE - Deve retornar BadRequest
/// </summary>
[Fact]
0 referências
public async Task UsuarioController_Delete_IdInseguro_DeveRetornarBadRequest()
{
    // Arrange
    var controller = CriarUsuarioController();

    // Act
    var resultado = await controller.Delete("usuario/../../admin");

    // Assert
    var badRequest = Assert.IsType<BadRequestObjectResult>(resultado);
    Assert.Equal(400, badRequest.StatusCode);
}

```

ResiduoController

- CT18 — GET Resíduo com ID Inseguro

Objetivo: Verificar se o controller de resíduos rejeita, com BadRequest, um ID inseguro na consulta.

```

/// <summary>
/// Cenário 18: ID inseguro no GET de resíduo - Deve retornar BadRequest
/// </summary>
[Fact]
0 referências
public async Task ResiduoController_GetById_IdInseguro_DeveRetornarBadRequest()
{
    // Arrange
    var controller = CriarResiduoController();

    // Act
    var resultado = await controller.GetById("../residuos");

    // Assert
    var badRequest = Assert.IsType<BadRequestObjectResult>(resultado);
    Assert.Equal(400, badRequest.StatusCode);
}

```

- CT19 — DELETE Resíduo com ID Inseguro

Objetivo: Verificar se o controller de resíduos rejeita, com BadRequest, um ID inseguro na exclusão.

```

/// <summary>
/// Cenário 19: ID inseguro no DELETE de resíduo - Deve retornar BadRequest
/// </summary>
[Fact]
0 referências
public async Task ResiduoController_Delete_IdInseguro_DeveRetornarBadRequest()
{
    // Arrange
    var controller = CriarResiduoController();

    // Act
    var resultado = await controller.Delete("residuos/../../admin");

    // Assert
    var badRequest = Assert.IsType<BadRequestObjectResult>(resultado);
    Assert.Equal(400, badRequest.StatusCode);
}

```

PontosColetaController

- CT20 — Sincronização com Cidade Vazia

Objetivo: Verificar se a sincronização é rejeitada, com BadRequest, quando o nome da cidade está vazio, sem acessar serviços externos.

```
/// <summary>
/// Cenário 20: Cidade vazia - Deve retornar BadRequest sem acessar serviços externos
/// </summary>
[Fact]
0 referências
public async Task PontosColetaController_SincronizarCidade_CidadeVazia_DeveRetornarBadRequest()
{
    // Arrange
    var controller = CriarPontosColetaController();

    // Act
    var resultado = await controller.SincronizarCidade("");

    // Assert
    var badRequest = Assert.IsType<BadRequestObjectResult>(resultado);
    Assert.Equal(400, badRequest.StatusCode);
}
```

- **CT21 — Sincronização com Cidade em Branco**

Objetivo: Verificar se a sincronização é rejeitada, com BadRequest, quando o nome da cidade contém apenas espaços em branco.

```
/// <summary>
/// Cenário 21: Cidade em branco - Deve retornar BadRequest
/// </summary>
[Fact]
0 referências
public async Task PontosColetaController_SincronizarCidade_CidadeEmBranco_DeveRetornarBadRequest()
{
    // Arrange
    var controller = CriarPontosColetaController();

    // Act
    var resultado = await controller.SincronizarCidade(" ");

    // Assert
    var badRequest = Assert.IsType<BadRequestObjectResult>(resultado);
    Assert.Equal(400, badRequest.StatusCode);
}
```

1.3 APPWPF — Services

Os 11 casos de teste a seguir verificam os serviços da aplicação desktop (WPF), cobrindo a validação/formatação de CPF e o gerenciamento do ciclo de vida da sessão do usuário logado. Código-fonte extraído da classe AppWpfServicesTests.cs.

ValidacaoCpfService

- **CT22 — CPF Válido sem Máscara**

Objetivo: Verificar se um CPF válido, informado sem máscara, é aceito.

```
/// <summary>
/// Cenário 22: CPF válido sem máscara - Deve retornar true
/// </summary>
[Fact]
0 referências
public void ValidarCpf_CpfValidoSemMascara_DeveRetornarTrue()
{
    // Arrange
    var cpf = "52998224725";

    // Act
    var resultado = ValidacaoCpfService.Validar(cpf);

    // Assert
    Assert.True(resultado);
}
```

- **CT23 — CPF Válido com Máscara**

Objetivo: Verificar se um CPF válido, informado com máscara, é aceito.

```
/// <summary>
/// Cenário 23: CPF válido com máscara - Deve retornar true
/// </summary>
[Fact]
0 referências
public void ValidarCpf_CpfValidoComMascara_DeveRetornarTrue()
{
    // Arrange
    var cpf = "529.982.247-25";

    // Act
    var resultado = ValidacaoCpfService.Validar(cpf);

    // Assert
    Assert.True(resultado);
}
```

- **CT24 — CPF Inválido**

Objetivo: Verificar a rejeição de um CPF com dígito verificador incorreto.

```
/// <summary>
/// Cenário 24: CPF inválido - Deve retornar false
/// </summary>
[Fact]
0 referências
public void ValidarCpf_CpfInvalido_DeveRetornarFalse()
{
    // Arrange
    var cpf = "529.982.247-26";

    // Act
    var resultado = ValidacaoCpfService.Validar(cpf);

    // Assert
    Assert.False(resultado);
}
```

- **CT25 — CPF com Dígitos Repetidos**

Objetivo: Verificar a rejeição de um CPF cujos dígitos são todos iguais.

```

/// <summary>
/// Cenário 25: CPF com todos os dígitos iguais - Deve retornar false
/// </summary>
[Fact]
0 referências
public void ValidarCpf_TodosDigitosIguais_DeveRetornarFalse()
{
    // Arrange
    var cpf = "111.111.111-11";

    // Act
    var resultado = ValidacaoCpfService.Validar(cpf);

    // Assert
    Assert.False(resultado);
}

```

- CT26 — CPF Vazio

Objetivo: Verificar o tratamento de um CPF vazio.

```

/// <summary>
/// Cenário 26: CPF vazio - Deve retornar false
/// </summary>
[Fact]
0 referências
public void ValidarCpf_CpfVazio_DeveRetornarFalse()
{
    // Arrange
    var cpf = "";

    // Act
    var resultado = ValidacaoCpfService.Validar(cpf);

    // Assert
    Assert.False(resultado);
}

```

- CT27 — Formatação de CPF

Objetivo: Verificar se a máscara é aplicada corretamente a um CPF válido sem formatação.

```

/// <summary>
/// Cenário 27: Formatação de CPF válido - Deve aplicar máscara
/// </summary>
[Fact]
0 referências
public void FormatarCpf_CpfValido_DeveAplicarMascara()
{
    // Arrange
    var cpf = "52998224725";

    // Act
    var resultado = ValidacaoCpfService.Formatar(cpf);

    // Assert
    Assert.Equal("529.982.247-25", resultado);
}

```

UsuarioSessaoService

- CT28 — Instância Única do Serviço de Sessão

Objetivo: Verificar se o serviço de sessão mantém uma única instância (padrão Singleton).

```
/// <summary>
/// Cenário 28: Singleton - Deve retornar a mesma instância
/// </summary>
[Fact]
0 referências
public void UsuarioSessaoService_Instancia_DeveSerSingleton()
{
    // Arrange / Act
    var instancia1 = UsuarioSessaoService.Instancia;
    var instancia2 = UsuarioSessaoService.Instancia;

    // Assert
    Assert.Same(instancia1, instancia2);
}
```

- CT29 — Início de Sessão

Objetivo: Verificar se, ao iniciar uma sessão, o usuário e seu ID são corretamente armazenados.

```
/// <summary>
/// Cenário 29: Iniciar sessão - Deve armazenar usuário e ID
/// </summary>
[Fact]
0 referências
public void IniciarSessao_UsuarioValido_DeveArmazenarUsuarioEId()
{
    // Arrange
    var service = UsuarioSessaoService.Instancia;
    var usuario = CriarUsuario();

    // Act
    service.IniciarSessao(usuario);

    // Assert
    Assert.Same(usuario, service.UsuarioLogado);
    Assert.Equal("usuario-teste-01", service.IdUsuarioLogado);

    // Cleanup
    service.EncerrarSessao();
}
```

- CT30 — Encerramento de Sessão

Objetivo: Verificar se, ao encerrar a sessão, os dados do usuário, o ID e a foto são limpos.

```

/// <summary>
/// Cenário 30: Encerrar sessão - Deve limpar usuário e foto
/// </summary>
[Fact]
0 referências
public void EncerrarSessao_SessaoAtiva_DeveLimparDados()
{
    // Arrange
    var service = UsuarioSessaoService.Instancia;
    service.IniciarSessao(CriarUsuario());
    service.FotoCaminho = "foto.jpg";

    // Act
    service.EncerrarSessao();

    // Assert
    Assert.Null(service.UsuarioLogado);
    Assert.Null(service.IdUsuarioLogado);
    Assert.Null(service.FotoCaminho);
}

```

- **CT31 — Expiração Forçada de Sessão**

Objetivo: Verificar se a expiração forçada limpa a sessão e dispara o evento correspondente.

```

/// <summary>
/// Cenário 31: Expiração forçada - Deve limpar sessão e disparar evento
/// </summary>
[Fact]
0 referências
public void ForcarExpiracaoSessao_SessaoAtiva_DeveDispararEvento()
{
    // Arrange
    var service = UsuarioSessaoService.Instancia;
    service.IniciarSessao(CriarUsuario());
    var expirou = false;
    service.SessaoExpirada += AoExpirar;

    // Act
    service.ForçarExpiracaoSessao();

    // Assert
    Assert.True(expirou);
    Assert.Null(service.UsuarioLogado);

    // Cleanup
    service.SessaoExpirada -= AoExpirar;

    void AoExpirar() => expirou = true;
}

```

- **CT32 — Notificação de Alteração do Usuário Logado**

Objetivo: Verificar se a alteração do usuário logado dispara as notificações de propriedade (PropertyChanged) esperadas.

```

/// <summary>
/// Cenário 32: Alteração de usuário - Deve notificar propriedades
/// </summary>
[Fact]
0 referências
public void UsuarioLogado_Alterado_DeveDispararPropertyChanged()
{
    // Arrange
    var service = UsuarioSessaoService.Instance;
    var propriedades = new List<string?>();
    PropertyChangedEventHandler handler = (_, args) =>
        propriedades.Add(args.PropertyName);

    service.PropertyChanged += handler;

    // Act
    service.UsuarioLogado = CriarUsuario();

    // Assert
    Assert.Contains(nameof(UsuarioSessaoService.UsuarioLogado), propriedades);
    Assert.Contains(nameof(UsuarioSessaoService.IdUsuarioLogado), propriedades);

    // Cleanup
    service.PropertyChanged -= handler;
    service.EncerrarSessao();
}

```

1.4 APPWPF — ViewModels

Os 10 casos de teste a seguir verificam a CadastroViewModel (máscara de CPF, controle de exibição do formulário e comandos de solicitação de acesso, validação de token e finalização de cadastro) e a BaseViewModel. Código-fonte extraído da classe AppWpfViewModelsTests.cs.

CadastroViewModel — CPF

- **CT33 — Aplicação de Máscara ao CPF**

Objetivo: Verificar se o CPF digitado sem máscara é formatado progressivamente pela ViewModel.

```

/// <summary>
/// Cenário 33: CPF digitado sem máscara - Deve aplicar máscara progressivamente
/// </summary>
[Fact]
0 referências
public void CadastroViewModel_CpfSemMascara_DeveAplicarMascara()
{
    // Arrange
    var viewModel = new CadastroViewModel();

    // Act
    viewModel.CPF = "52998224725";

    // Assert
    Assert.Equal("529.982.247-25", viewModel.CPF);
}

```

- **CT34 — Limite de Dígitos do CPF**

Objetivo: Verificar se a ViewModel limita a entrada do CPF a 11 dígitos numéricos.

```

/// <summary>
/// Cenário 34: CPF acima de 11 dígitos - Deve limitar a 11 números
/// </summary>
[Fact]

public void CadastroViewModel_CpfComMaisDe11Digitos_DeveLimitar()
{
    // Arrange
    var viewModel = new CadastroViewModel();

    // Act
    viewModel.CPF = "529982247259999";

    // Assert
    Assert.Equal("529.982.247-25", viewModel.CPF);
}

```

CadastroViewModel — Estado do formulário

- CT35 — Token Inválido Oculta o Formulário

Objetivo: Verificar se, com token inválido, o formulário completo de cadastro permanece oculto.

```

/// <summary>
/// Cenário 35: Token inválido - Estado inicial não exibe formulário completo
/// </summary>
[Fact]
0 referências
public void CadastroViewModel_TokenInvalido_DeveOcultarFormularioCompleto()
{
    // Arrange
    var viewModel = new CadastroViewModel();

    // Act
    viewModel.IsTokenValido = false;

    // Assert
    Assert.False(viewModel.ExibirFormularioCompleto);
}

```

- CT36 — Token Válido Exibe o Formulário

Objetivo: Verificar se, com token válido, o formulário completo de cadastro é exibido.

```

/// <summary>
/// Cenário 36: Token válido - Deve exibir formulário completo
/// </summary>
[Fact]
0 referências
public void CadastroViewModel_TokenValido_DeveExibirFormularioCompleto()
{
    // Arrange
    var viewModel = new CadastroViewModel();

    // Act
    viewModel.IsTokenValido = true;

    // Assert
    Assert.True(viewModel.ExibirFormularioCompleto);
}

```

- CT37 — Cadastro Finalizado Oculta o Formulário

Objetivo: Verificar se, após a finalização do cadastro, o formulário completo volta a ficar oculto.

```
/// <summary>
/// Cenário 37: Cadastro finalizado - Deve ocultar formulário completo
/// </summary>
[Fact]
0 referências
public void CadastroViewModel_CadastroFinalizado_DeveOcultarFormularioCompleto()
{
    // Arrange
    var viewModel = new CadastroViewModel
    {
        IsTokenValido = true
    };

    // Act
    viewModel.CadastroFinalizadoComSucesso = true;

    // Assert
    Assert.False(viewModel.ExibirFormularioCompleto);
}
```

CadastroViewModel — Comandos e validações

- CT38 — Solicitação de Acesso sem E-mail

Objetivo: Verificar se a ausência de e-mail impede a solicitação de acesso e gera mensagem de obrigatoriedade.

```
/// <summary>
/// Cenário 38: Solicitar acesso sem e-mail - Deve apresentar mensagem de validação
/// </summary>
[Fact]

public void SolicitarAcesso_EmailVazio_DeveInformarObrigatoriedade()
{
    // Arrange
    var viewModel = new CadastroViewModel();

    // Act
    Assert.True(viewModel.SolicitarAcessoCommand.CanExecute(null));
    viewModel.SolicitarAcessoCommand.Execute(null);

    // Assert
    Assert.Equal("O e-mail é obrigatório.", viewModel.MensaEmail);
}
```

- CT39 — Solicitação de Acesso com E-mail Inválido

Objetivo: Verificar se um e-mail em formato inválido gera mensagem de validação de formato.

```
/// <summary>
/// Cenário 39: Solicitar acesso com e-mail inválido - Deve apresentar mensagem
/// </summary>
[Fact]
0 referências
public void SolicitarAcesso_EmailInvalido_DeveInformarFormato()
{
    // Arrange
    var viewModel = new CadastroViewModel
    {
        Email = "email-invalido"
    };

    // Act
    viewModel.SolicitarAcessoCommand.Execute(null);

    // Assert
    Assert.Equal("Informe um endereço de e-mail válido.", viewModel.MensaEmail);
}
```

- **CT40 — Validação sem Token**

Objetivo: Verificar se a ausência do token de ativação gera mensagem de obrigatoriedade.

```
/// <summary>
/// Cenário 40: Validação sem token - Deve solicitar o token
/// </summary>
[Fact]
0 referências
public void ValidarToken_TokenVazio_DeveInformarObrigatoriedade()
{
    // Arrange
    var viewModel = new CadastroViewModel
    {
        Email = "teste@regraphik.com"
    };

    // Act
    viewModel.ValidarTokenCommand.Execute(null);

    // Assert
    Assert.Equal(
        "Por favor, insira o token de ativação.",
        viewModel.MensagemErroToken);
}
```

- **CT41 — Finalização sem Dados Obrigatórios**

Objetivo: Verificar se a ausência de nome, login, senha e CPF gera as respectivas mensagens de validação ao finalizar o cadastro.

```
/// <summary>
/// Cenário 41: Finalização sem dados obrigatórios - Deve preencher mensagens de validação
/// </summary>
[Fact]
0 referências
public void FinalizarCadastro_DadosObrigatoriosAusentes_DeveInformarErros()
{
    // Arrange
    var viewModel = new CadastroViewModel();

    // Act
    viewModel.FinalizarCadastroCommand.Execute(null);

    // Assert
    Assert.Equal("O nome completo é obrigatório.", viewModel.MensaNome);
    Assert.Equal("O login é obrigatório.", viewModel.MensaLogin);
    Assert.Equal("A senha é obrigatória.", viewModel.MensaSenha);
    Assert.Equal("O CPF é obrigatório.", viewModel.MensaCpf);
}
```

BaseViewModel

- **CT42 — Interface INotifyPropertyChanged**

Objetivo: Verificar se a ViewModel base implementa a interface INotifyPropertyChanged, base para o binding de dados.

```

/// <summary>
/// Cenário 42: ViewModel base - Deve implementar INotifyPropertyChanged
/// </summary>
[Fact]
0 referências
public void BaseViewModel_DeveImplementarINotifyPropertyChanged()
{
    // Arrange
    var viewModel = new BaseViewModel();

    // Act
    var resultado = viewModel is System.ComponentModel.INotifyPropertyChanged;

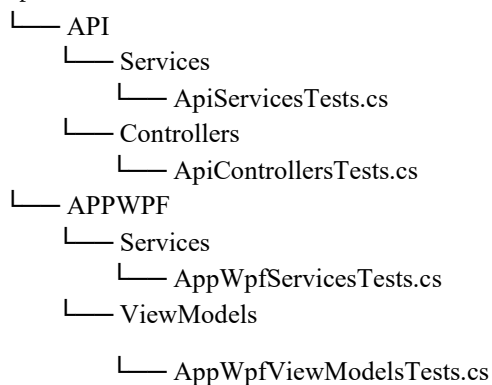
    // Assert
    Assert.True(resultado);
}

```

2 ESTRUTURA DOS TESTES

Os testes foram implementados em quatro classes específicas do projeto de testes, mantendo a separação entre o código de produção e o código destinado à verificação automatizada.

ReGraphik.Testes



Cada classe reúne os testes referentes às regras do componente escolhido, organizados em regiões (#region) por método/controller/serviço testado. As asserções são utilizadas de acordo com o comportamento esperado de cada método.

3 DOCUMENTAÇÃO DA ESTRATÉGIA UTILIZADA

A estratégia foi baseada na seleção de componentes com métodos determinísticos e testáveis de forma isolada, cobrindo tanto a camada de API (Services e Controllers) quanto a camada de aplicação desktop APPWPF (Services e ViewModels). Foram definidos cenários positivos e negativos, além de entradas nulas, vazias, valores limites e entradas relacionadas à segurança.

A execução foi realizada por meio do xUnit integrado ao Visual Studio. Cada caso possui uma condição de entrada e uma expectativa objetiva, permitindo que uma falha seja identificada automaticamente. Nos controllers, dependências externas (como HttpClient e configuração) foram substituídas por implementações de teste, isolando a verificação da regra de validação de entrada.

- Isolamento do componente testado.
- Independência entre os casos de teste.
- Uso de Arrange, Act e Assert.
- Uso de asserções compatíveis com cada cenário.
- Validação de entradas válidas e inválidas.
- Verificação de valores limites.
- Verificação de respostas HTTP (BadRequest) em controllers da API.

- Análise dos resultados após a execução.

4 EXPLICAÇÃO DOS CASOS DE TESTE

Os casos foram definidos para representar situações relevantes ao comportamento dos componentes selecionados. A combinação dos 42 cenários permite verificar diferentes caminhos de execução das camadas de API e APPWPF.

4.1 Validação de identificadores (API)

Os casos CT01 a CT06 verificam identificadores válidos, nulos, vazios, contendo tentativa de Path Traversal, caracteres inválidos e tamanho superior ao limite.

4.2 Sanitização de textos (API)

Os casos CT07 a CT09 verificam o tratamento de conteúdo HTML, texto nulo e texto acima do limite. Durante a execução, o CT07 apresentou inicialmente uma expectativa incompatível com o comportamento real do método.

4.3 Validação de resíduos (API)

Os casos CT10 e CT11 verificam, respectivamente, uma entrada válida e uma entrada com campos obrigatórios inválidos.

4.4 Validação de usuários (API)

Os casos CT12 a CT15 verificam usuário válido, e-mail inválido, login abaixo do tamanho mínimo e senha abaixo do tamanho mínimo.

4.5 Controllers da API

Os casos CT16 a CT21 verificam se UsuarioController, ResiduoController e PontosColetaController retornam BadRequest diante de IDs inseguros (tentativas de path traversal) e de parâmetros obrigatórios vazios ou em branco, sem acionar dependências externas.

4.6 Validação e formatação de CPF (APPWPF)

Os casos CT22 a CT27 verificam a validação de CPFs válidos (com e sem máscara), CPFs inválidos, CPFs com dígitos repetidos, CPF vazio e a formatação de um CPF válido.

4.7 Gerenciamento de sessão do usuário (APPWPF)

Os casos CT28 a CT32 verificam o padrão Singleton do serviço de sessão, o início e o encerramento de sessão, a expiração forçada e o disparo do evento PropertyChanged ao alterar o usuário logado.

4.8 ViewModel de cadastro (APPWPF)

Os casos CT33 a CT41 verificam a máscara e o limite de dígitos do CPF, o controle de exibição do formulário completo conforme o estado do token e do cadastro, e as mensagens de validação dos comandos de solicitação de acesso, validação de token e finalização de cadastro.

4.9 ViewModel base (APPWPF)

O caso CT42 verifica se a BaseViewModel implementa a interface INotifyPropertyChanged, requisito para o data binding das telas WPF.

5 RESULTADO FINAL DOS TESTES

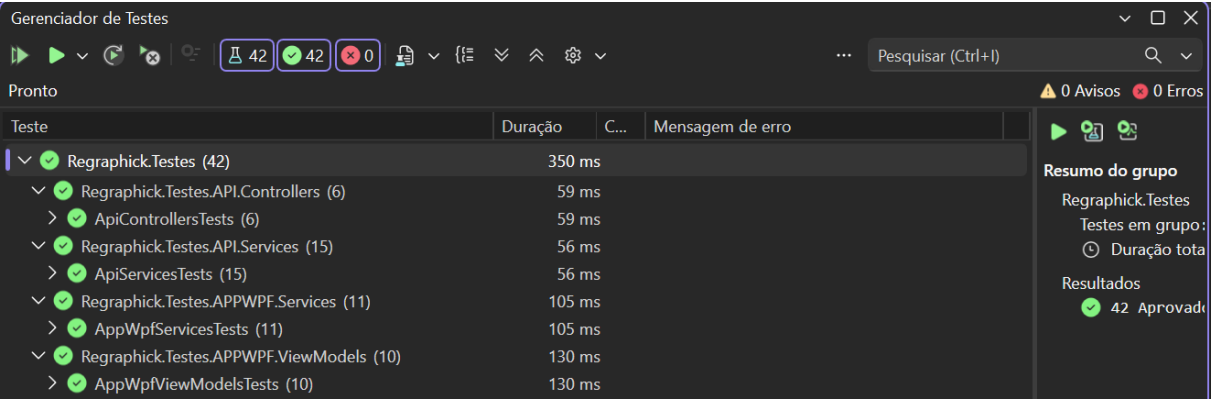


Figura 1 — Teste executado com Aprovação.

Fonte: Elaborado pelo autor (2026).

Na execução inicial do teste de sanitização HTML (CT07), foi identificada uma diferença entre o resultado esperado e o resultado produzido pelo método.

```
Expected: "Olá Bruno"
Actual: "Olá alert('x') Bruno"
```

A análise mostrou que o método remove as tags HTML, mas preserva o conteúdo textual que estava dentro delas. Por esse motivo, a expectativa do teste foi ajustada para representar o comportamento efetivo do componente.

```
Assert.Equal("Olá alert('x') Bruno", resultado);
```

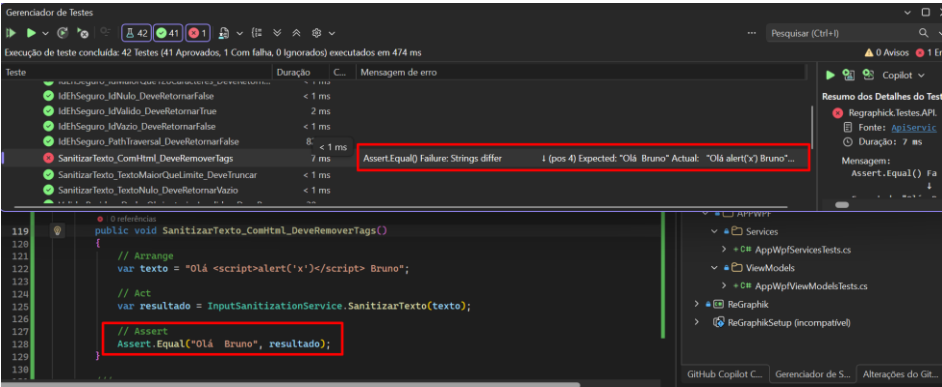


Figura 2 — Falha inicial do teste de sanitização HTML.

Fonte: Elaborado pelo autor (2026).

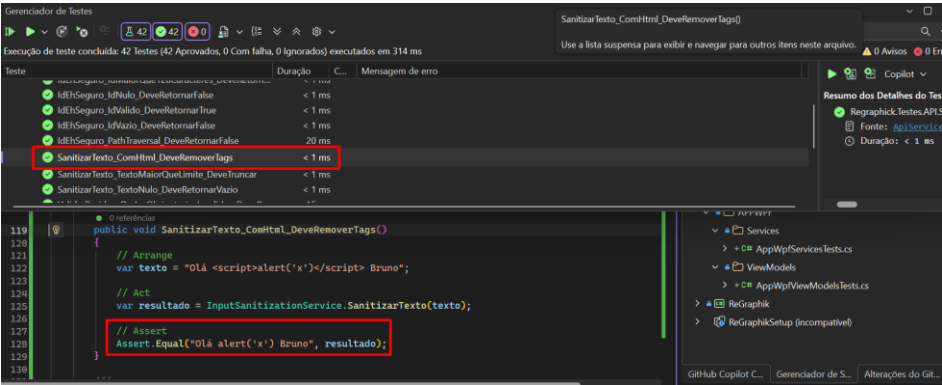


Figura 3 — Código do teste após o ajuste da expectativa.

Fonte: Elaborado pelo autor (2026).

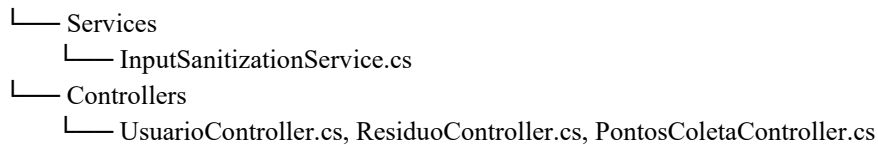
Após o ajuste da asserção, os testes foram executados novamente e apresentaram resultado aprovado. O registro da execução final deve ser utilizado como evidência principal da conclusão da suíte.

6 IDENTIFICAÇÃO CLARA DOS COMPONENTES ESCOLHIDOS

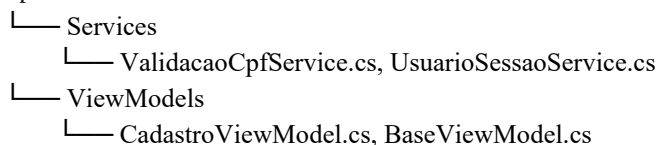
Os componentes escolhidos para a realização da atividade foram os seguintes, distribuídos entre a API e a aplicação desktop (APPWPF) do projeto ReGraphik:

- InputSanitizationService — camada Services da API.
- UsuarioController, ResiduoController e PontosColetaController — camada Controllers da API.
- ValidacaoCpfService e UsuarioSessaoService — camada Services do APPWPF.
- CadastroViewModel e BaseViewModel — camada ViewModels do APPWPF.

ApiRestReGraphik



ReGraphikAPPWPF



As classes de teste responsáveis pela verificação desses componentes são a ApiServiceTests.cs, a ApiControllerTests.cs, a AppWpfServicesTests.cs e a AppWpfViewModelsTests.cs, localizadas no projeto ReGraphik.Testes.

7 EVIDÊNCIA DE CONTRIBUIÇÃO NO PROJETO

A contribuição individual é demonstrada pela implementação da suíte de testes para os componentes escolhidos, pela execução e análise dos resultados e pelo registro das alterações no repositório do projeto.

Para garantir rastreabilidade, recomenda-se inserir uma captura do histórico de commits contendo as alterações referentes à criação ou atualização da suíte de testes nas quatro classes.

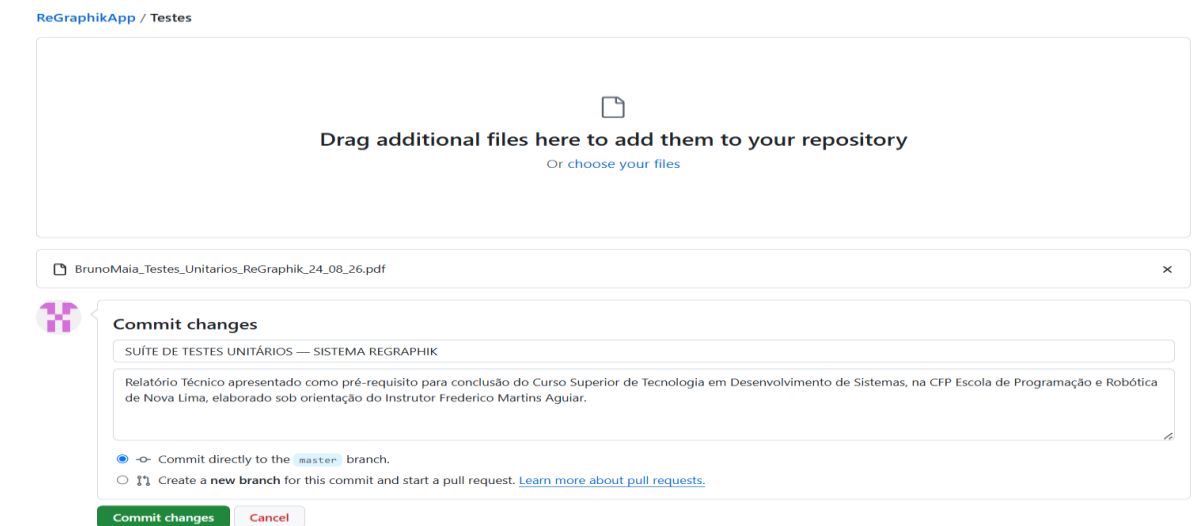


Figura 3 — Histórico do GitHub demonstrando o commit relacionado à contribuição individual.
Fonte: Elaborado pelo autor (2026).

A evidência deve permitir relacionar o aluno, os componentes, as classes de teste e as alterações realizadas no projeto.

9 CONSIDERAÇÕES FINAIS

A atividade permitiu aplicar os conceitos de testes unitários no projeto de TCC ReGraphik, estabelecendo uma suíte composta por 42 casos distribuídos em quatro classes de teste, cobrindo os componentes InputSanitizationService, UsuarioController, ResiduoController, PontosColetaController, ValidacaoCpfService, UsuarioSessaoService, CadastroViewModel e BaseViewModel, nas camadas API e APPWPF do sistema.

Além da implementação dos testes, foi realizada a análise dos resultados. A falha encontrada no teste de sanitização demonstrou a importância de comparar a expectativa definida no teste com o comportamento efetivamente implementado antes de modificar o código de produção.

A documentação e as evidências apresentadas permitem demonstrar a contribuição individual para a qualidade e a confiabilidade do projeto, tanto na API quanto na aplicação desktop.

10 CHECKLIST DE EVIDÊNCIAS

Tabela 1 — Checklist de entrega da atividade

Item	Evidência	Status
Suíte de testes unitários desenvolvida	42 casos documentados (4 classes)	Concluído
Código dos testes	ApiServicesTests.cs, ApiControllersTests.cs, AppWpfServicesTests.cs, AppWpfViewModelsTests.cs	Concluído
Evidências da execução	Capturas do Test Explorer	Concluído
Documentação da estratégia	Seção 4	Concluído
Explicação dos casos de teste	Seção 5	Concluído
Resultado final dos testes	Execução após ajuste (CT07)	Concluído
Componentes escolhidos	8 componentes (API e APPWPF)	Concluído
Contribuição no projeto	Histórico/commit do GitHub	Concluído

Fonte: Elaborado pelo autor (2026).